TI-2600 Laboratório de Desenvolvimento Multiplataforma

Antonio Carvalho - Treinamentos

Model View View Model (MVVM)

ViewModel – O que é ?

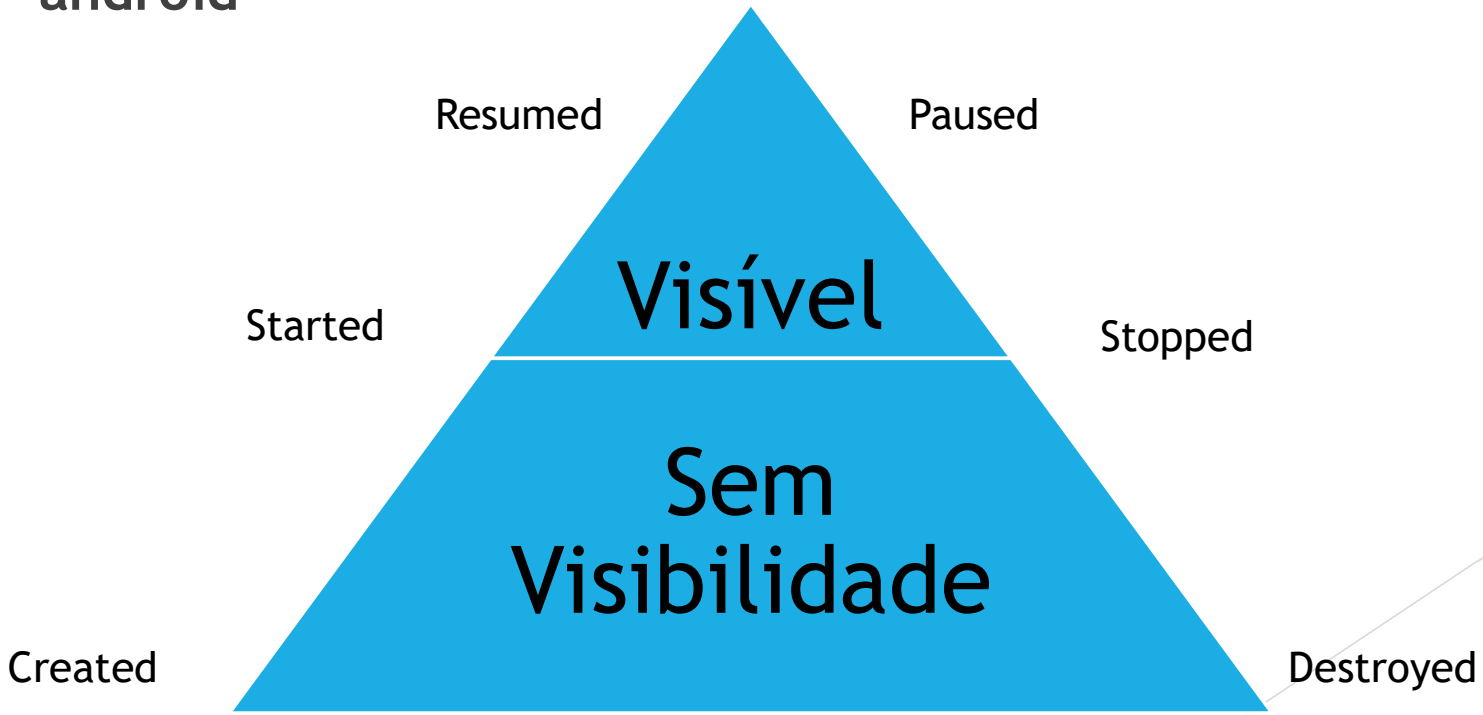
- A ViewModel é uma camada de lógica de negócios que pode ser acoplada a uma Activity.
- Esta camada atua como um **controlador** guardando o **estado atual** das informações
- Há uma importância na separação dos códigos referentes a tela e lógica de negócio, isto facilita a manutenibilidade da aplicação.
- Este conceito é conhecido como **separation of concerns (SoC).**
- **SoC** inclui dois conceitos principais: **acoplamento** e **coesão**

ViewModel – O que é ?

- Ela expõe o estado para interface do usuário e encapsula a lógica de negócios relacionada.
- Sua principal vantagem é que ele armazena em cache o estado e o persiste por meio de alterações de configuração.
- Isso significa que a interface do usuário não precisa buscar dados novamente ao navegar entre atividades ou ao seguir **alterações de configuração**, como ao girar a tela. (No caso do Android)
- O escopo de vida do **ViewModel** é maior que o da Activity ou das telas do **Compose**

ViewModel – O que é ?

- **Visibilidade da aplicação no ciclo de vida da activity no android**



ViewModel – O que é ?

- A vida do ViewModel permeia todos os ciclos da Activity, ou seja ela guarda informações que persistem pelas mudanças de configurações como exemplo:
  - Tamanho de exibição do aplicativo
  - Orientação da tela
  - Tamanho e peso da fonte
  - Localidade
  - Modo escuro versus modo claro
  - Disponibilidade do teclado
- Quando ocorre uma destas mudanças o **Android** recria a **Activity** invocando o **onDestroy()** e o **onCreate()** novamente

ViewModel - Instalação

- Para instalar o **ViewModel** é preciso alterar o arquivo **libs.versions.toml**
- É importante garantir que a dependência **kotlin-coroutines-swing** esteja implementada também para assegurar que o **ViewModelScope** possa existir no **Desktop**

```
[versions]

...

kotlinx-coroutines = "1.10.1"

androidx-lifecycle = "2.8.4"


[libraries]

...

androidx-lifecycle-viewmodel = { group = "org.jetbrains.androidx.lifecycle",  name =
"lifecycle-viewmodel", version.ref = "androidx-lifecycle" }

androidx-lifecycle-runtime-compose = { group = "org.jetbrains.androidx.lifecycle", name =
"lifecycle-runtime-compose", version.ref = "androidx-lifecycle" }

androidx-lifecycle-viewmodel-compose = { module =
"org.jetbrains.androidx.lifecycle:lifecycle-viewmodel-compose", version.ref = "androidx-
lifecycle" }

kotlinx-coroutines-swing = { group = "org.jetbrains.kotlinx", name = "kotlinx-coroutines-
swing", version.ref = "kotlinx-coroutines" }
```

ViewModel - Instalação

- Em seguida é preciso modificar o arquivo **`build.gradle.kts`** do módulo **composeApp**, adicionando a seguinte **dependencia**

```
kotlin {
    // ...
    sourceSets {
        // ...
           commonMain.dependencies {
            // ...
                 implementation(libs.androidx.lifecycle.viewmodel)
                 implementation(libs.androidx.lifecycle.runtime.compose)
                 implementation(libs.androidx.lifecycle.viewmodel.compose)
           }
           desktopMain.dependencies {
                 implementation(libs.kotlinx.coroutines.swing)
        }
        // ...
    }
}
```

ViewModel - Implementação

- Para a implementação é preciso criar uma classe que herde de **ViewModel**

```
import androidx.compose.runtime.IntState
import androidx.compose.runtime.MutableIntState
import androidx.compose.runtime.mutableIntStateOf
import androidx.lifecycle.ViewModel

class ContadorViewModel(startValue : Int) : ViewModel() {
    init {
        // O que executar quando o ViewModel for instanciado
    }
    private val _contador : MutableIntState =
                mutableIntStateOf( startValue )
    val contador : IntState = _contador
    fun incrementCounter() {
        _contador.value++
    }
}
```

ViewModel - Implementação

- Usar o ViewModel dentro de um Composable

```
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
@Preview
fun App() {
       MaterialTheme {
           val contadorViewModel : ContadorViewModel = viewModel {
                  ContadorViewModel(0) }
           ContadorComponente(contadorViewModel )
       }
}
@Composable
fun ContadorComponent( contadorViewModel : ContadorViewModel ) {
    Text( "Contador: ${contadorViewModel.contador.value}")
    Button( onClick = { contadorViewModel.incrementCounter() } ) {
        Text( "Incrementar" )
    }
}
```